BONTEN Event Management System – Architecture Overview

In developing BONTEN, our web-based event management platform, we focused on building a system that is organized, secure, and responsive. Using PHP, MySQL, and JavaScript, we implemented a three-tier architecture inspired by the Model-View-Controller (MVC) pattern. This approach allowed us to clearly separate concerns, manage complexity, and build a platform that can evolve with future enhancements.

Technology Stack

- Backend: PHP 7.4+
- Database: MySQL (via MySQLi)
- Frontend: HTML5, CSS3, JavaScript (ES6+)
- Payment Integration: Paystack API
- Security: CSRF protection, rate limiting, prepared statements

MVC Pattern & Layered Structure

BONTEN follows an MVC-inspired architecture, effectively separating presentation from business logic, while using a lightweight approach for data access.

Model Layer:
We implemented a centralized `Database.php` class to handle queries directly from controllers. Although we did not create dedicated Model classes for every entity, this approach allowed us to develop efficiently while keeping data access organized. It also lays a foundation for future refactoring toward a fully structured Model layer.

View Layer:
Our views manage the user interface and dynamic content. User dashboards, event pages, and forms are designed with PHP embedded in HTML, keeping the interface responsive and functional while receiving clean data from controllers. This separation has helped us maintain clarity and consistency across the platform.

Controller Layer:
Controllers manage requests, business logic, and database interactions. They handle event creation, ticket sales, payment processing, RSVPs, reviews, and comments. API endpoints follow RESTful principles, ensuring modularity and scalability. This structure has allowed us to coordinate functionality effectively across the system.

Three-Tier Architecture

Presentation Layer (Client-Side):
Using `public/js` and `public/css`, this layer manages dynamic updates, AJAX/Fetch calls, and client-side validation. JavaScript modules handle event interactions, dashboards, multi-step forms, and comment systems, creating a smooth and interactive experience for users.

Application/Logic Layer (Server-Side):
Controllers and embedded logic process requests, enforce authentication and authorization, and execute business rules. They manage events, bookings, payments, and user content, ensuring data validation and session integrity.

Data Layer (Database):
The MySQL database stores users, events, tickets, RSVPs, reviews, comments, and bookings. Relationships and transactions maintain data integrity, while cascading foreign keys simplify updates and deletions.

Key Features

- User & Manager Management: Secure authentication, role-based access, and profile updates. We ensured a balance between security and usability.

- Event & Content Management: Multi-step event creation, ticket management, and dynamic listings. Structuring these features clearly improved our development workflow.

- Dynamic Interactions: AJAX-based reviews, comments, RSVP management, and dashboards provide real-time feedback, improving user engagement.

- Security: CSRF tokens, prepared statements, and secure headers helped us enforce practical web security standards.

Reflection

Through BONTEN, we learned the value of thoughtful architecture and teamwork. Separating responsibilities across the three layers made development more efficient and maintenance more straightforward. Implementing dynamic features, secure authentication, and integrated payment workflows reinforced how design decisions directly impact user experience. Overall, building BONTEN has been a collaborative, reflective process that strengthened our understanding of system design, security, and usability.